

National University of Computer and Emerging Sciences



Lab Manual 11 Programming Fundamentals

Course Instructor	Ms. Anosha Khan
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Fall 2025

FAST - School of Computing
FAST-NU, Lahore Campus

Activities:

Note: Include input validation in programs wherever applicable. Don't use any built-in functions.

1. Write a program that performs the following operations on a 2D character matrix. The matrix must be square such as 3x3, 4x4, etc. For this problem, you may assume it is 5x5.
 - Initialize a square matrix of characters.
 - Create a function that takes a 2D char array consisting of 5 columns as a parameter, replaces all vowels in the matrix with '?', and leaves the consonants unchanged.
 - Create another function that rotates the modified matrix by 90 degrees clockwise. You may use another temporary array of size 5x5 in this function. However, the original matrix must be updated/rotated.
 - A function that prints any 2D char array.

Function Prototypes:

- void replaceVowels(char matrix[][5], int rows);
- void rotateMatrix90Clockwise(char matrix[][5], int rows);
- void printMatrix(const char matrix[][5] int rows);

Example Input, shown for 3x3 but your program should work for 5x5:

Enter matrix:

A B C

D E F

G H I

Example Output:

After the replace function:

? B C

D ? F

G H ?

Rotated Matrix:

G D ?

H ? B

? F C

2. Write a program that processes a 2D character matrix and performs the following operations. For this problem, you may assume the array is ROWx5.
 - Function **searchWord**: Search a given word in the matrix. The word can appear horizontally [right to left] or vertically [top to down]. If the word is found, return true, its starting and ending positions otherwise return false. Be careful not to go past the array boundaries at [0] and [4]. This function must call two other functions: **searchHorizontal** and **searchVertical**. For example, the function **searchHorizontal** will take as parameters a 2D char array **data**, 1D char array representing a **word**,

indices **i** & **j**, and look for the word in **data** from right-to-left starting from the position **[i][j]**. If the word is found it returns true.

- **Replace the Found Word:** Replace all occurrences of the word in the matrix with a specified replacement word. If the replacement word is shorter than the original word, pad the remaining cells with '*'. If it is longer, truncate the replacement word to fit the space. This function must call the function **searchWord**.
- **Display Matrices:** Print any 2D char array with 5 columns.
- **Print:** Print the matrix, highlight the positions of the found word, and then display the matrix after replacements.

Function Prototypes:

- `bool searchWord(const char data[][5], int rows, char word[], int& startRow, int& startCol, int& endRow, int& endCol);`
 - `bool searchHorizontal(const char data[][5], int rows, char word[], int i, int j)`
 - `bool searchVertical( ... )`.
- `void FindnReplace(char data[][5], int rows, const char word[], const char replacement[])`. This function must call the function **searchWord**. Hint: `searchWord` will calculate 4 values: `startRow`, `startCol`, `endRow` and `endCol`. Use this to determine the orientation of the word (horizontal or vertical).
- `void printMatrix(const char data[][5], int rows);`

Example Input

Matrix:

```
C  X  Y  Z
T  A  C  F
G  H  I  J
```

Word to find: "CAT"

Replacement: "FOX"

Example Output

Found Word Positions:

Start: (1, 2), End: (1, 0)

Modified Matrix:

```
C  X  Y  Z
X  O  F  F
G  H  I  J
```

3. Write a program to define the function

```
void trim(char matrix[][20], int R);
```

This function takes a 2D char array matrix of size $R \times 20$ (R rows, with each row being a null-terminated string) and removes any leading and trailing spaces from each row in place. The function should call two helper functions:

- `int leadingSpaces(char row[]);` // Returns the count of leading spaces in the given row.
- `int trailingSpaces(char row[]);` // Returns the count of trailing spaces in the given row.

The trim function should modify the matrix directly, ensuring no additional arrays or strings are used. After trimming, any row that initially consisted only of spaces should be replaced by an empty string ("").

Example:

Input:

```
matrix [[20] = {
    "  hello world  ",
    "   example   ",
    "      test      ",
    "    ",
    "nothingToTrim"
};
```

Output:

```
matrix [[20] = {
    "hello world",
    "example",
    "test",
    "",
    "nothingToTrim"
};
```

4. Write user defined functions to accomplish the following task:
Creating a 2D array B from a two-dimensional array A, with size N rows and N+1 columns such that it will store the elements skipping the elements below the main diagonal of the array A and filling the remaining elements with Null character.

Example:

Array A

a	b	c	d	e	'\0'
f	g	h	i	j	'\0'
k	l	m	n	o	'\0'
p	q	r	s	t	'\0'
u	v	w	x	y	'\0'

Array B

a	b	c	d	e	'\0'
g	h	i	j	'\0'	'\0'
m	n	o	'\0'	'\0'	'\0'
s	t	'\0'	'\0'	'\0'	'\0'
y	0	'\0'	'\0'	'\0'	'\0'

You must write three functions:

- Function 1 to take input in array A.
- Function 2 to copy the elements of the array A to the array B.
- Function 3 to display elements of the array A and array B.